

Chronologie Détaillée du Handoff

Phase 1 : Navigation Normale (Shard A est le maître)

Acteurs actifs : Client, Broker, Shard A.

Flux : Le Client envoie ses MoveInput au Broker. Le Broker les route uniquement vers le Shard A.

État interne : Le Shard A possède l'entité avec le statut Owned. Il calcule la physique et publie l'AOI. Le Shard B ne sait même pas que ce joueur existe.

Phase 2 : Entrée dans la Marge du shard B

Le Déclencheur : Le joueur s'approche de la frontière. Le Spatial Server s'en rend compte grâce aux PositionUpdate et envoie : **CrossingAlert {client_id, shard_A_topic, Shard_B_topic }**.

La Réaction du Réseau (Broker) :

- Subscribe le **Shard_B_topic** aux **inputs** du client (pendant un moment les inputs client seront drop par le shard B, tant qu'il n'a pas reçu l'ordre de faire spawn l'entité en ghost)
- Subscribe le client aux **publish** du **Shard_B_topic**
- Il route le message au **shard_A_topic** seulement

Le Transfert d'État :

- Le Shard A reçoit la CrossingAlert. Envoie un **HandoffRequest (0x20)** contenant la position exacte, la vitesse, etc. empaqueté dans un **InterShardMessage**, directement à **Shard_B_topic**.
- Le Shard A passe son state pour le client de **Owned** à **PendingHandoff** en stockant **Shard_B_topic** dedans.
- Le Shard B reçoit ce **0x20** et fait spawner le joueur chez lui, mais avec le statut **Ghost**.

La Boucle de Synchronisation :

Le Shard B simule le joueur grâce aux inputs clonés qu'il reçoit. Mais pour être sûr qu'il ne désynchronise pas, le Shard A (qui est toujours l'Autorité) lui envoie des **GhostUpdate (0x23) via InterShardMessage** à chaque tick pour le recadrer si besoin.

Côté Client : Le client reçoit toujours son image du shard A mais reçoit également celle du shard B.

(L'image du shard B ne contient que les acteur **Owned**, donc ne contient rien concernant le client lui même, aucune concurrence entre les deux shards)

Phase 3 : Le Franchissement (Le "Switch" d'Autorité)

Le Déclencheur : Le joueur traverse la ligne mathématique exacte de la frontière. Le Spatial Server envoie **AuthoritySwitch { client_ID, Shard_A_Topic, Shard_B_Topic }** à destination du Shard A SEULEMENT (Ici le broker ne fait que relayer l'info).

La Bascule (Inversion des rôles) :

- Le Shard A (qui était maître) envoie le message **HandoffComplete (0x24)** au Shard B pour lui donner l'autorité. (Tjrs via **InterShardMessage**)
- Le Shard B reçoit le **0x24** et passe de **Ghost** à **PendingHandoff** en stockant le **Shard_A_Topic** dedans. Il commence à publier le joueur dans son AOI.
- Le Shard A dégrade le client de **PendingHandoff** à **Ghost**. Il arrête de publier le joueur dans son AOI et reçoit maintenant des **GhostUpdates**. Par contre il continue à envoyer son image au client (pour que le client puisse toujours connaître la position des autres joueurs)

Côté Client : Il recevait sa position validée dans l'AOI du Shard A, maintenant c'est l'AOI du Shard B qui prend le relais pour afficher ce personnage. Aucun changement visible ou logique

Phase 4 : Sortie de la Marge (Nettoyage du Shard A)

Le Déclencheur : Le joueur s'enfonce profondément dans le territoire du Shard B. Il n'est plus proche de la frontière. Le Spatial Server envoie: **CrossingExit { client_ID, obsolete_Shard_A, new_shard_B }**.

La Réaction du Réseau (Broker) :

- Unsubscribe le Shard A des **inputs** client
- Unsubscribe le client du **publish** du shard A
- Route le message **CrossingExit** aux DEUX shards (A et B)

La Fermeture :

- Le shard A reçoit le **CrossingExit** (son topic est en première position, c'est donc une suppression pour lui), il supprime l'entité client de son registre
- Le shard B reçoit le **CrossingExit** (son topic est en deuxième position, c'est donc un ajout pour lui), il a 2 choix :
 - Soit il y a d'autres topic que **obsolete_Shard_A** dans la liste des topics liés au **PendingHandoff** et dans ce cas il ne touche à rien

- Soit il n'y a plus d'autre topic que le **obsolete_Shard_A** et dans ce cas il passe l'entité en **owned**

Nouvelle Navigation Normale : On est de retour à la Phase 1, mais avec le Shard B aux commandes.

(Liste des messages page suivante)

A. SpatialServer vers Shards

CROSSING_ALERT (Tag: 0x25) {client_id: u32, dest_authority_topic: [u8; 32], neighbor_topic: [u8; 32]} :

- **Envoyé par** : Service Spatial
- **Action du broker au routage** :
 - Fait suivre le message au server dont le topic est **dest_authority_topic**.
 - Parallèlement, abonne la shard **neighbor_topic** aux ClientInput.
 - Et abonne le client aux publish du **neighbor_topic**
- **Action du Shard Autorité à la réception** :
 - Passe l'état de l'entité visée en PendingHandoff et génère un HandoffRequest à destination du neighbor_topic.

AUTHORITY_SWITCH (Tag: 0x26) { client_id: u32, old_auth_topic: [u8; 32], new_auth_topic: [u8; 32]} :

- **Envoyé par** : Service Spatial
- **Action du broker au routage** : Relais le message à **old_auth_topic**.
- **Action des Shards à la réception** :
 - L'ancien Shard (old) dégrade l'entité de **Owned** à **Ghost** (il arrête de l'inclure dans son AOI public).
 - Le nouveau Shard (new) promeut l'entité de **Ghost** à **PendingHandoff** (il commence à l'inclure dans son AOI public).

CROSSING_EXIT (Tag: 0x27) { client_id: u32, obsolete_auth_topic: [u8; 32], new_auth_topic: [u8; 32]} :

- **Envoyé par** : Service Spatial
- **Action du broker au routage** :
 - Unsubscribe **obsolete_auth_topic** des **inputs** client
 - Unsubscribe le client du **publish** de **obsolete_auth_topic**
 - Route le message aux deux shards
- **Action des shards à la réception** :
 - Nettoyage du côté du old shard
 - Switch des states côté new shared (soumis à conditions)

B. Flux Inter shards

Seul nouveau match du Broker :

BrokerMessage::InterShardMessage (Tag: **0x30**) { topic_dest : [u8; 32], topic_from : [u8; 32], payload : [u8] }

Le broker ne fait que router ces messages directement vers le topic destination

Communications imbriquées dans le payload précédent :

HANDOFF_REQUEST (Tag: **0x20**) { entity_id: u32, pos: Vec2, vel: Vec2, state: [u8; 64] } :

- **Envoyé par** : Shard Autorité
- **Action du broker au routage** : Lit le topic de destination sur l'enveloppe et transfère le message au Shard voisin (Fantôme).
- **Action du Shard Fantôme à la réception** : Fait spawnner une nouvelle entité locale avec les données fournies et lui attribue le tag **Ghost**.

GHOST_UPDATE (Tag: **0x23**) { entity_id: u32, pos: Vec2, vel: Vec2 } :

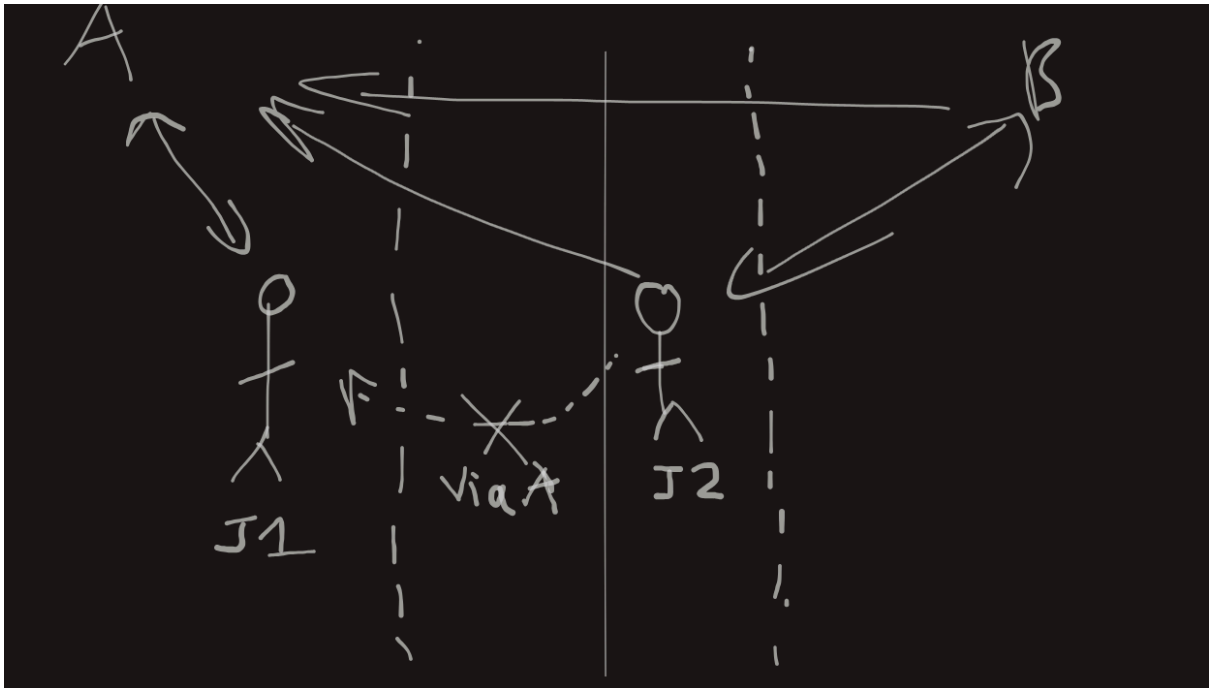
- **Envoyé par** : Shard Autorité (à chaque tick)
- **Action du broker au routage** : Transfère au Shard Fantôme (via le flux *Unreliable*).
- **Action du Shard Fantôme à la réception** : Compare sa propre simulation locale de l'entité avec la **pos** et **vel** reçues. Si la prédiction locale est fausse, le Fantôme écrase sa position avec celle du message pour rester parfaitement synchronisé.

HANDOFF_COMPLETE (Tag: **0x24**) { entity_id: u32 } :

- **Envoyé par** : Le shard Autorité à la réception du switch
- **Action du broker au routage** : Transfère au Shard destinataire (la nouvelle autorité).
- **Action de la nouvelle autorité à la réception** : Passe son entité de **ghost** à **PendingHandoff**.

Problèmes probables :

- Est-ce que 2 joueurs qui traversent dans le même sens se voient : **oui**, ils viennent de la même shard
- Est-ce que 2 joueurs qui traversent dans des sens opposés se voient : **oui**, car ils sont simulés par la shard de destination (l'un pour l'autre), donc la shard destination leur envoie la position du joueur dans son AOI
- Est-ce qu'un joueur proche d'une marge voit les joueurs qui traversent arriver ? Probablement pas.



Le joueur du shard A ne verra le joueur du shard B apparaître que à l'instant où J2 cross la hard limite, ou à l'instant où J1 cross la marge de B.

Solution : faire des marges suffisamment grand pour couvrir le champ de vision du joueur (en prenant en compte la vitesse)

- Est-ce qu'un joueur qui traverse vers une shard (A vers B) peut voir les joueurs qui traversent de C vers B ? Sans doute qu'en ajustant les marges on peut gérer ça

